Departamento de Engenharia
Eletrónica e Telecomunicações
e de Computadores

Circuitos Aritméticos e Lógicos

Relatório do 2º Trabalho

Trabalho realizado por:
Gustavo Costa | Nº 52808
Rafael Pereira | Nº 52880

Turma: **LEIC15D**
Docente: **Prof.º Mário Véstias**

Licenciatura em Engenharia Informática e de Computadores
Lógica e Sistemas Digitais (LSD)
2024 / 2025 - Semestre de Inverno

Conteúdo

Índice **i**

1 Introdução **5**

1.1 Descrição do Trabalho 5

1.2 Objetivos 6

2 Análise e Projeto **7**

2.1 Operações Aritméticas 7

2.1.1 Adição 7

2.1.2 Subtração 7

2.1.3 Incremento 8

2.1.4 Decremento 8

2.2 Operações Lógicas 8

2.2.1 Deslocamento Lógico à Direita (LSR) 8

2.2.2 Deslocamento Aritmético à Direita (ASR) 9

2.2.3 Deslocamento Lógico à Esquerda (LSL) 9

2.2.4 Operação NAND 9

2.3 Geração e Descrição das Flags 10

2.3.1 *Flag* CBo (*Carry/Borrow*) 10

2.3.2 *Flag* OV (*Overflow*) 10

2.3.3 *Flag* Z (Zero) 10

2.3.4 *Flag* P (Paridade) 10

2.3.5 *Flag* GE (*Greater or Equal*) 10

2.3.6 *Flag* BE (*Below or Equal*) 10

2.4 Diagramas Lógicos 11

2.4.1 Módulo Aritmético 11

2.4.2 Módulo Lógico 12

2.4.3 Módulo de *Flags* 13

2.4.4 Módulo *Decoder* 13

2.4.5 Módulo MUX (ALU) 13

2.4.6 Módulo Y*or0* 14

2.4.7 ALU (Entidade Topo) 14

3 Montagem Laboratorial **15**
3.1 Configuração dos pinos 15
3.2 Testes para Operações Aritméticas 16
3.2.1 Teste 21 - Configuração e Resultado 16
3.2.2 Teste 22 - Configuração e Resultado 16
3.3 Testes para Operações Lógicas 17
3.3.1 Teste 23 - Configuração e Resultado 17
3.3.2 Teste 24 - Configuração e Resultado 17
3.4 Comparação dos Resultados 18
3.4.1 Resultados Teóricos *versus* Experimentais 18
3.4.2 Análise da Consistência e Erros 18

4 Conclusão **19**
4.1 Considerações sobre o trabalho desenvolvido 19
4.2 Discussão de Resultados 19

Referências **20**

A VHDL **21**
A.1 Módulo Aritmético 21
A.1.1 Somador/Subtrator 22
A.1.2 Somador de 4 Bits 23
A.1.3 Somador de 1 Bit 24
A.1.4 Flags (AU) 24
A.2 Módulo Lógico 25
A.2.1 ASR 26
A.2.2 LSR 26
A.2.3 LSL 26
A.2.4 NAND 27
A.2.5 MUX (LU) 27
A.3 Decoder 27
A.4 Flags (ALU) 28
A.5 Yor0 28
A.6 MUX (ALU) 28
A.7 ALU 29
A.8 Test bench 31

B Esquema Elétrico **34**

Lista de Figuras

1 Expressão da Adição . . . 7
2 Expressão da Subtração . . . 7
3 Expressão do Incremento . . . 8
4 Expressão do Decremento . . . 8
5 Expressão do LSR . . . 8
6 Expressão do ASR . . . 9
7 Expressão do LSL . . . 9
8 Expressão do operador NAND . . . 9
9 Diagrama lógico do Módulo Aritmético . . . 11
10 Sub-Módulos do Módulo Aritmético . . . 11
11 Diagrama lógico do Módulo Lógico . . . 12
12 Sub-Módulos do Módulo Lógico . . . 12
13 Diagrama lógico do Módulo *FLAGS* . . . 13
14 Diagrama lógico do Módulo *DECODER* . . . 13
15 Diagrama lógico do Módulo MUX (ALU) . . . 13
16 Diagrama lógico do Módulo Y$or0$ . . . 14
17 Diagrama lógico da ALU . . . 14
18 Código de atribuição de pinos . . . 15
19 Teste de um incremento . . . 16
20 Teste de uma subtração . . . 16
21 Teste de um deslocamento aritmético à direita . . . 17
22 Teste de um deslocamento lógico à esquerda . . . 17
23 Tabela completa de resultados . . . 18
24 Código do Módulo Aritmético . . . 21
25 Código do Somador/Subtrator . . . 22
26 Código Somador de 4 Bits . . . 23
27 Código Somador de 1 Bit . . . 24
28 Código Flags (AU) . . . 24
29 Código do Módulo Lógico . . . 25
30 Código do ASR . . . 26
31 Código LSR . . . 26
32 Código LSL . . . 26
33 Código NAND . . . 27
34 Código MUX (LU) . . . 27

35 Código Decoder 27
36 Código Flags (ALU) 28
37 Código Yor0 28
38 Código MUX (ALU) 28
39 Código ALU (Parte 1) 29
40 Código ALU (Parte 2) 30
41 Código Test Bench (Parte 1) 31
42 Código Test Bench (Parte 2) 32
43 Código Test Bench (Parte 3) 33
44 Esquema elétrico da ALU 34

1 Introdução

1.1 Descrição do Trabalho

Este trabalho de laboratório consiste no desenvolvimento de uma Unidade Aritmética e Lógica (ALU) em VHDL, testada na placa *DE10-Lite* da *Intel*. A motivação deste projeto é compreender e aplicar conceitos fundamentais de arquitetura de sistemas digitais, manipulação de lógica binária e design modular, reforçando habilidades em programação de hardware e análise de resultados.

A ALU foi projetada para executar operações aritméticas e lógicas com operandos de 4 *bits* e gerar *flags* que indicam condições específicas dos resultados. As operações incluem adição, subtração, incremento, decremento, deslocamentos (*shift*) e uma operação NAND, todas controladas por uma entrada de 3 *bits*. Todas as operações foram realizadas em complemento de dois, garantindo o suporte para números negativos dentro do circuito.

Para facilitar o desenvolvimento, a ALU foi dividida em módulos específicos, os quais foram posteriormente integrados em um módulo principal. Como ferramenta de desenvolvimento e simulação, foi utilizado o software *Quartus*, que possibilitou a verificação do comportamento do circuito ao nível da simulação antes da implementação física na *DE10-Lite*. A funcionalidade do circuito foi testada com entradas específicas e comparada a valores teóricos fornecidos pelo enunciado.

Desafios enfrentados incluíram a integração dos módulos e a implementação dos sinais de *flags* com precisão. Foram aplicadas técnicas de depuração e simulação de *waveforms* para resolver esses problemas, garantindo que o circuito responderia corretamente às diversas combinações de entrada.

1.2 Objetivos

O objetivo deste trabalho é consolidar conhecimentos de Lógica de Sistemas Digitais e aplicar design modular para integrar funções numa *ALU*. Através da simulação e implementação prática, pretende-se:

- Desenvolver competências em *VHDL* estrutural, incluindo a criação de módulos que simulam operações aritméticas, lógicas e de geração de *flags*;
- Integrar o design modular num sistema completo, construído a partir de unidades básicas que executam operações específicas;
- Testar e validar a *ALU* usando a placa *DE10-Lite*, comprovando o funcionamento do circuito através de entradas configuradas e da análise dos resultados;
- Analisar a precisão do circuito através da comparação entre resultados teóricos e experimentais, garantindo que a implementação prática corresponde com o previsto;
- Aplicar, devida e corretamente, a usabilidade do programa *Quartus*.

2 Análise e Projeto

2.1 Operações Aritméticas

As operações aritméticas realizadas pela ALU incluem somas e subtrações entre operandos de 4 *bits*, além de operações de incremento e decremento. Essas operações utilizam um *bit* adicional, *carry-in* (CBi), que permite expandir a funcionalidade aritmética da ALU ao incorporar o conceito de "vai-um" em somas e "empréstimo" em subtrações.

2.1.1 Adição

A operação de adição soma os valores dos operandos X e Y, incluindo o valor do *carry-in* (CBi), resultando num número de 4 bits. Se o resultado ultrapassar o valor máximo representável em 4 *bits*, ocorre um *carry-out*, indicado pela *flag* CBo. A *flag* OV (overflow) é ativada quando o resultado excede a capacidade de representação de números com sinal, indicando um erro de estouro de valor. A adição pode ser representada logicamente pela expressão:

$$\boxed{X + Y + CBi = R}$$

Figura 1: Expressão da Adição

2.1.2 Subtração

A operação de subtração calcula a diferença entre os operandos X e Y, considerando o bit *carry-in* (CBi) como *borrow-in* ou empréstimo. Essa operação gera as mesmas *flags* que a adição, com uma diferença: a *flag* CBo aqui sinaliza um *borrow-out*, indicando que foi necessário um "empréstimo" para a operação. A *flag* OV indica ativa quando o resultado está fora dos limites de representação de números com sinal. A expressão lógica da subtração é dada por:

$$\boxed{X - Y - CBi = R}$$

Figura 2: Expressão da Subtração

2.1.3 Incremento

A operação de incremento adiciona o valor de X ao *carry-in* (CBi), aumentando o operando em uma unidade quando $CBi = 1$. Durante o incremento, o bloco $Yor0$ é ativado, desconsiderando o valor de Y (todos os *bits* de Y são configurados para zero). O incremento gera as mesmas *flags* que a adição, com exceção de que o *carry-in* é adicionado diretamente ao operando X. A expressão lógica que representa o incremento é:

$$X + CBi = R$$

Figura 3: Expressão do Incremento

2.1.4 Decremento

A operação de decremento subtrai o *carry-in* (CBi) de X, diminuindo o valor do operando em uma unidade, caso $CBi = 1$. Nesta operação, o bloco $Yor0$ também é ativado, eliminando a influência do operando Y. As *flags* geradas pelo decremento são as mesmas da subtração, com a *flag* CBo indicando um *borrow-out*, se houver necessidade de "empréstimo" durante a operação. A expressão lógica para o decremento é:

$$X - CBi = R$$

Figura 4: Expressão do Decremento

2.2 Operações Lógicas

Além das operações aritméticas, a ALU é projetada para realizar operações lógicas com o X (e com Y caso seja um NAND). O Módulo Lógico, para além de retornar o resultado das operações, retorna o bit que desaparece nas operações de deslocamento (LSR, ASR, LSL).

2.2.1 Deslocamento Lógico à Direita (LSR)

O deslocamento lógico à direita é a operação binária que move todos os 4 *bits* uma posição para a direita, preenchendo a posição restante na extremidade esquerda com zero. A expressão lógica para o deslocamento lógico à direita é:

$$X >>> 1$$

Figura 5: Expressão do LSR

2.2.2 Deslocamento Aritmético à Direita (ASR)

Diferentemente do deslocamento lógico à direita, o deslocamento aritmético à direita é uma operação binária que move todos os *bits* de um número para a direita por uma certa quantidade de posições, mas preservando o *bit* de sinal (o *bit* mais à esquerda). A expressão lógica para o deslocamento aritmético à direita é:

$$X >> 1$$

Figura 6: Expressão do ASR

2.2.3 Deslocamento Lógico à Esquerda (LSL)

O deslocamento lógico à esquerda é uma operação binária que move todos os *bits* de um determinado número para a esquerda por uma certa quantidade de posições, preenchendo a posição restante na extremidade esquerda com zero. A expressão lógica para o deslocamento lógico à esquerda é:

$$X <<< 1$$

Figura 7: Expressão do LSL

2.2.4 Operação NAND

O operador lógico NAND é uma operação booleana que utiliza os operadores AND ($\cdot$) e NOT ($\neg$). Em resumo, esse operador lógico resulta no complemento da operação AND e é representado pela expressão:

$$\overline{X \cdot Y}$$

Figura 8: Expressão do operador NAND

2.3 Geração e Descrição das Flags

2.3.1 *Flag* CBo (*Carry/Borrow*)

Esta *flag* representa o *carry* de saída da operação de soma ou o *borrow* de saída da operação de subtração. Fica ativa quando o resultado excede o domínio. No âmbito das operações lógicas (LSR, ASR e LSL), a *flag* retorna o valor do *bit* deslocado de X que desaparece do resultado.

2.3.2 *Flag* OV (*Overflow*)

Sempre que o resultado de uma operação excede o domínio dos números relativos a *flag* de *overflow* é ativada.

2.3.3 *Flag* Z (Zero)

Esta *flag* fica ativa quando o resultado de uma operação é zero (0).

2.3.4 *Flag* P (Paridade)

Quando o resultado de uma operação tem um número ímpar de 1s a *flag* é ativada.

2.3.5 *Flag* GE (*Greater or Equal*)

Esta *flag* tem a função de indicar se o primeiro operando (X), é maior ou igual ao segundo operando ($Y + CBi$). A *flag* atua na representação de números relativos e só deve ser considerada quando a operação realizada é uma subtração.

2.3.6 *Flag* BE (*Below or Equal*)

Similarmente à *flag* anterior, a *flag* BE tem a função de indicar se o primeiro operando (X), é menor ou igual ao segundo operando ($Y + CBi$). Ao contrário da *flag* GE, esta *flag* atua na representação de números naturais e só deve ser considerada quando a operação realizada é uma subtração.

2.4 Diagramas Lógicos

2.4.1 Módulo Aritmético

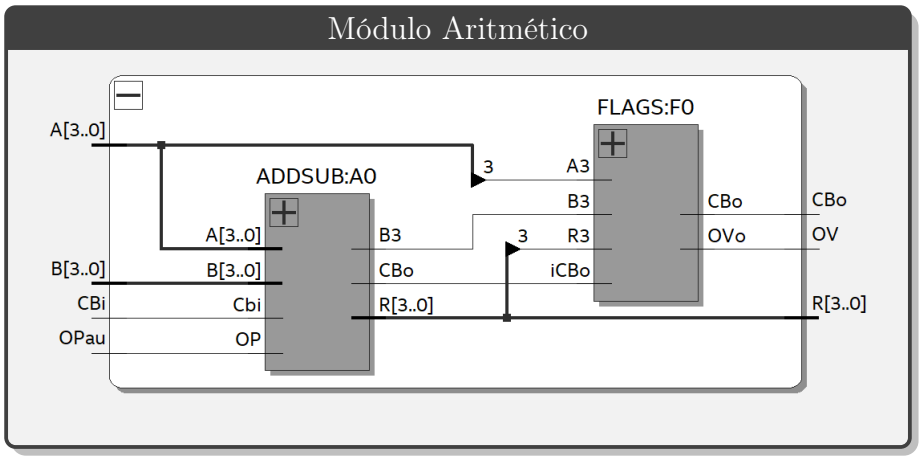

Figura 9: Diagrama lógico do Módulo Aritmético

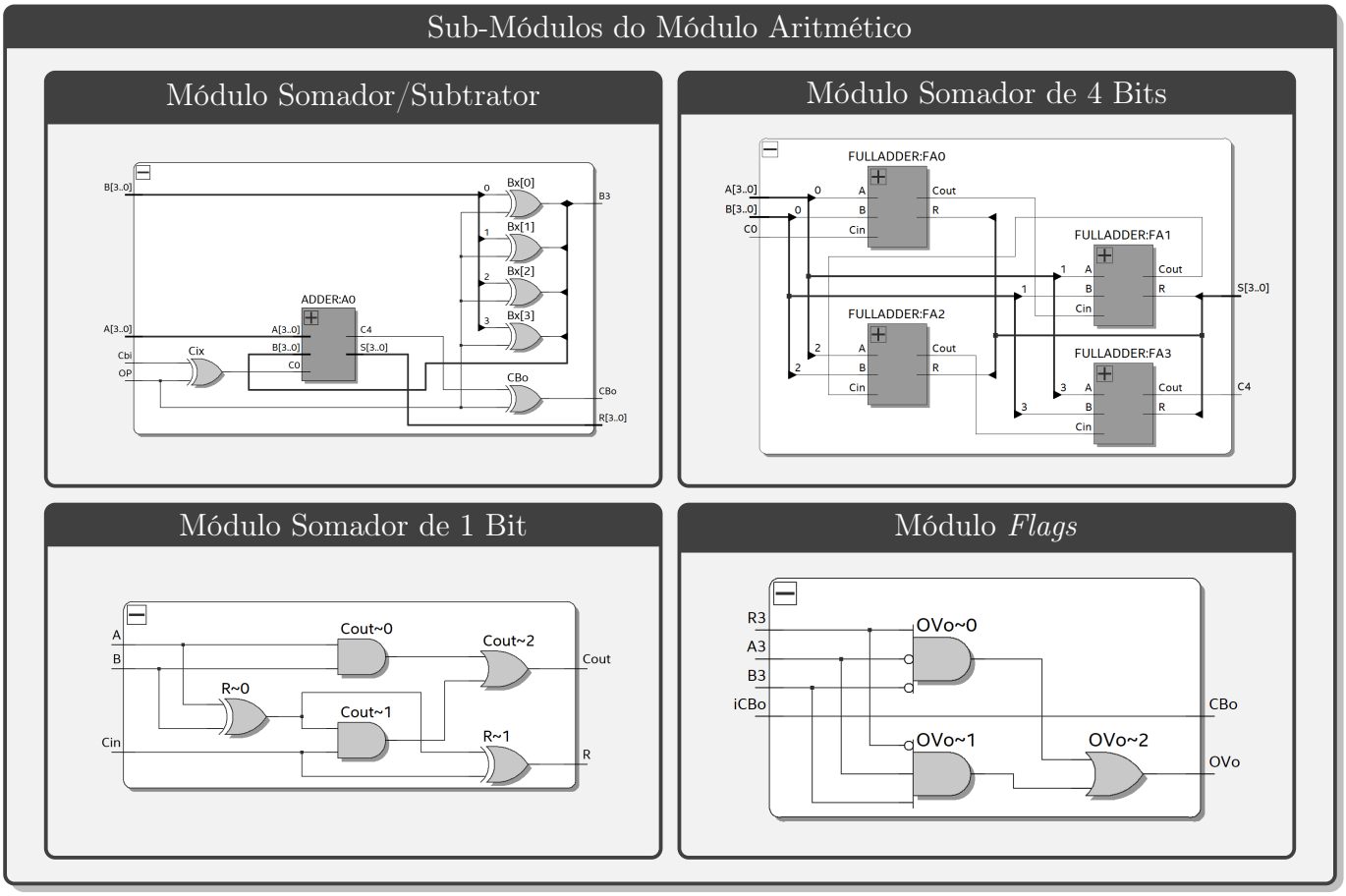

Figura 10: Sub-Módulos do Módulo Aritmético

2.4.2 Módulo Lógico

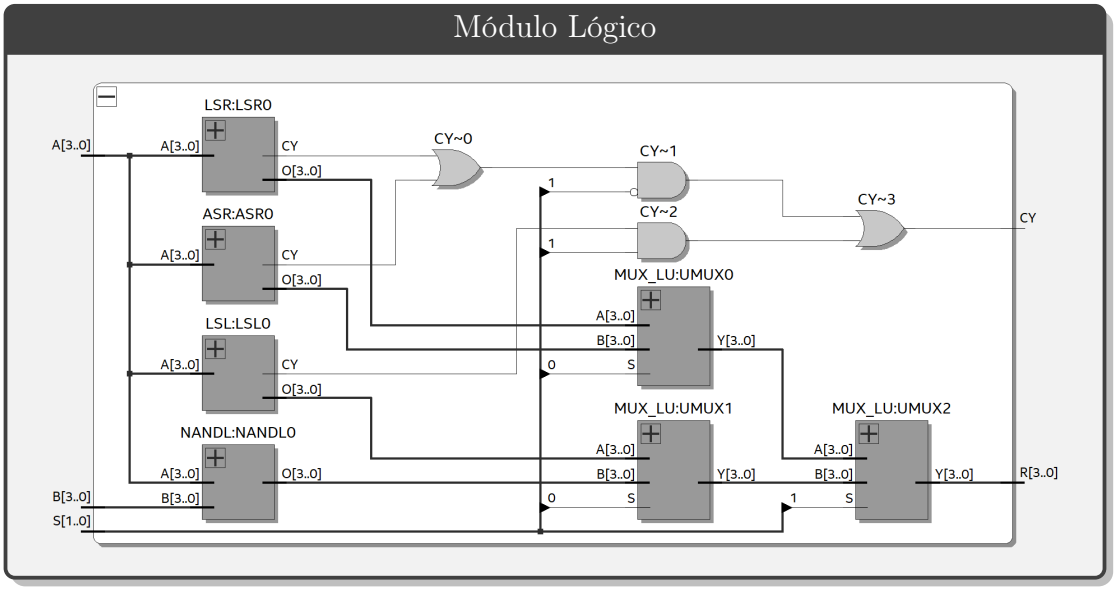

Figura 11: Diagrama lógico do Módulo Lógico

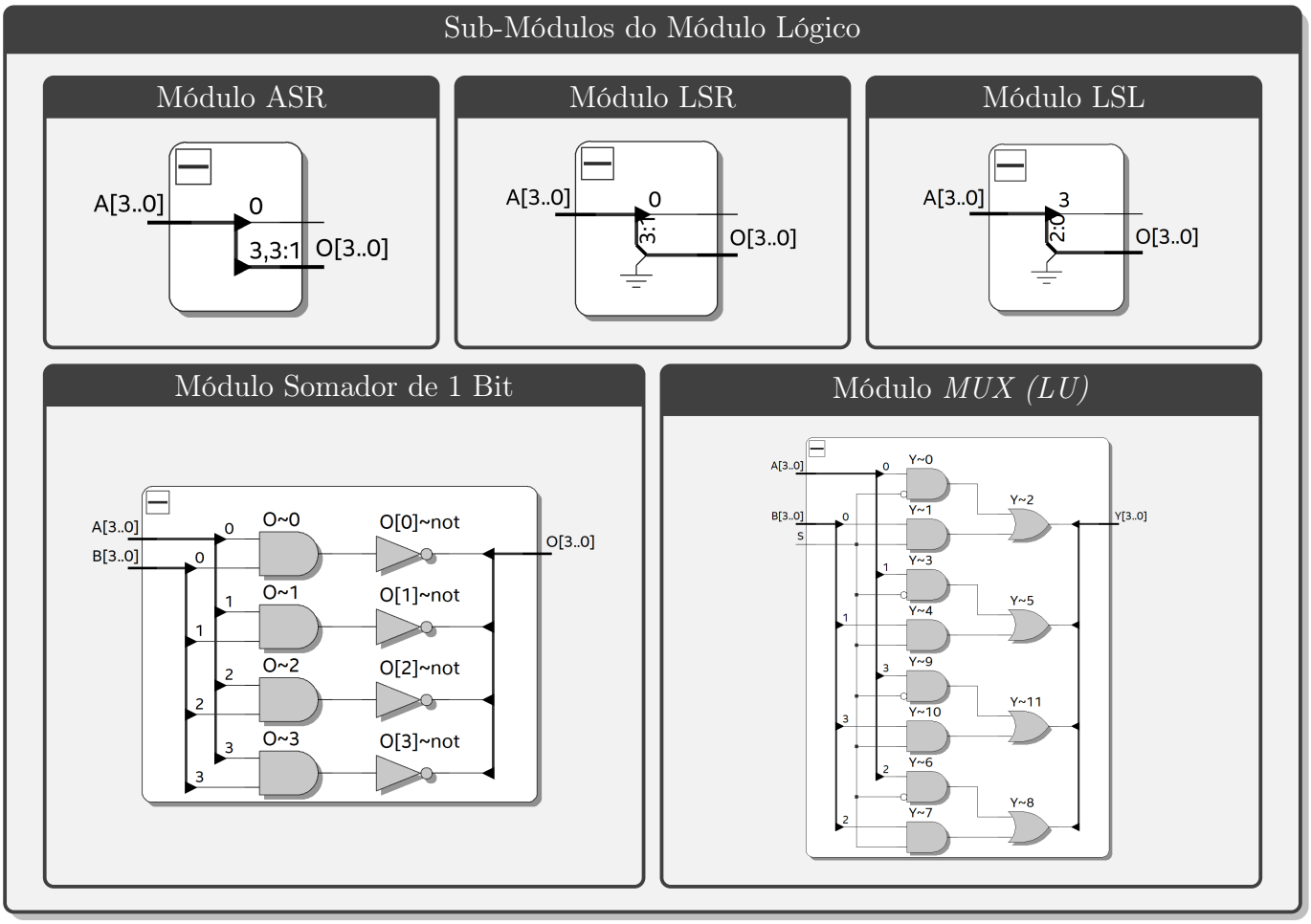

Figura 12: Sub-Módulos do Módulo Lógico

2.4.3 Módulo de *Flags*

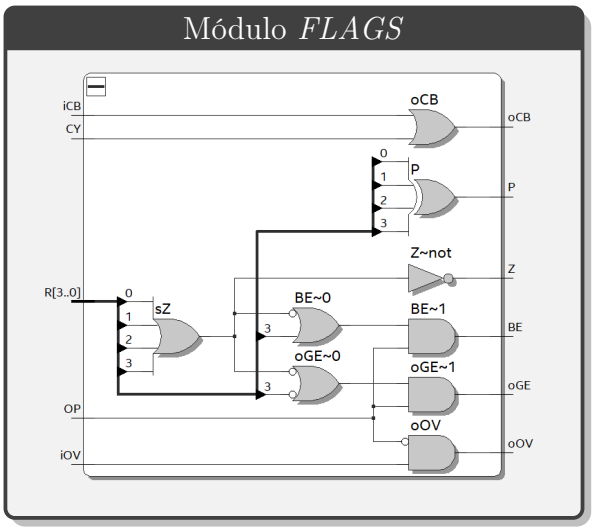

Figura 13: Diagrama lógico do Módulo *FLAGS*

2.4.4 Módulo *Decoder*

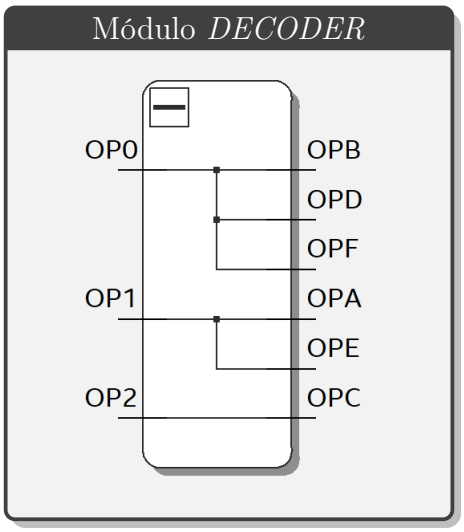

Figura 14: Diagrama lógico do Módulo *DECODER*

2.4.5 Módulo MUX (ALU)

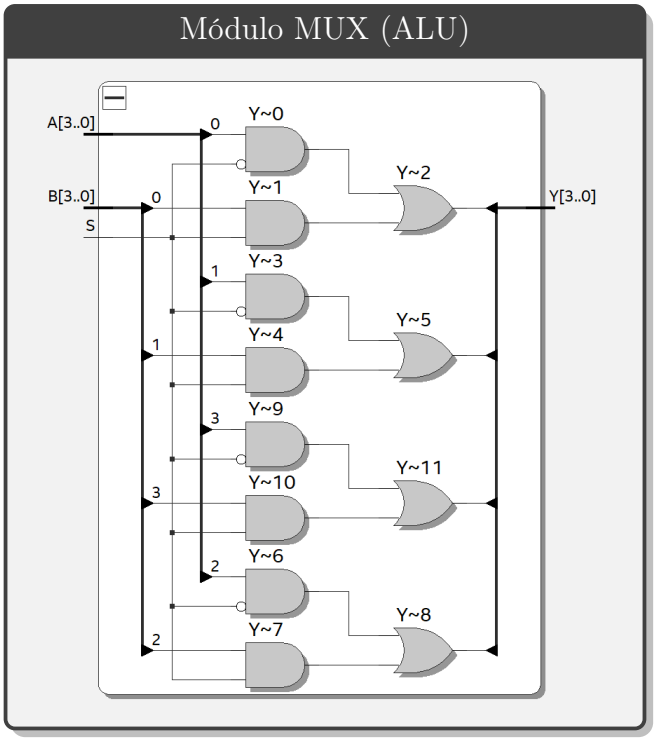

Figura 15: Diagrama lógico do Módulo MUX (ALU)

2.4.6 Módulo Y*or0*

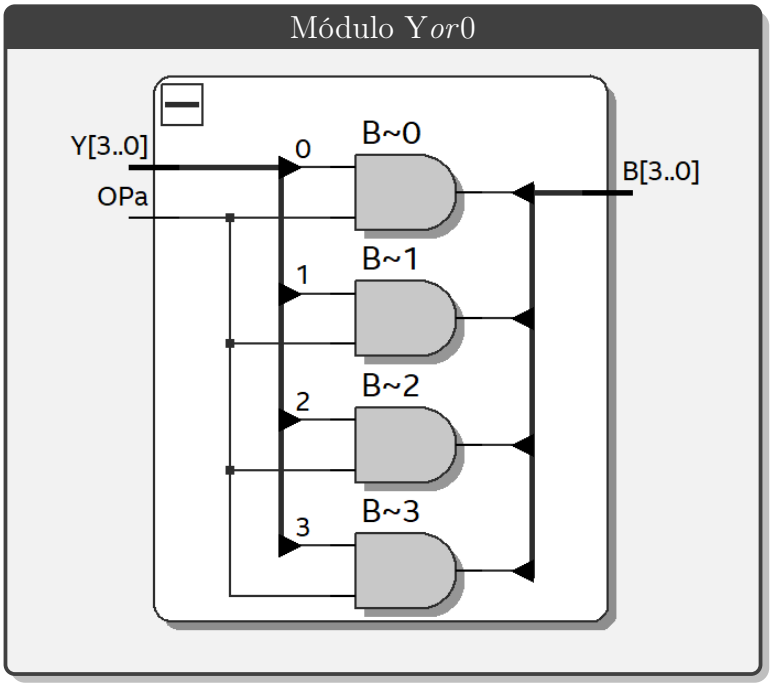

Figura 16: Diagrama lógico do Módulo Y*or*0

2.4.7 ALU (Entidade Topo)

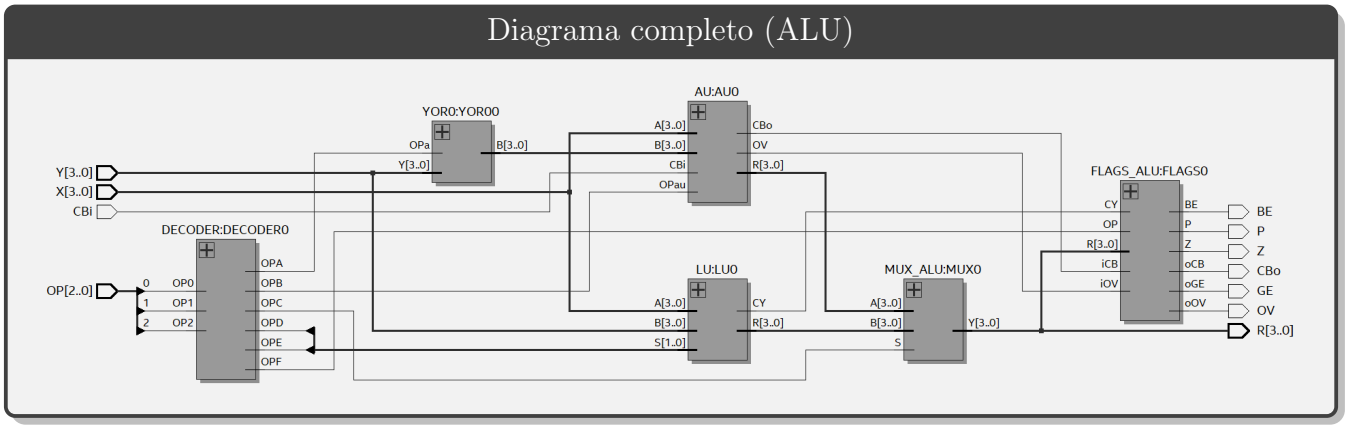

Figura 17: Diagrama lógico da ALU

3 Montagem Laboratorial

3.1 Configuração dos pinos

Utilizando, como base, o manual da placa *DE10-Lite* procedemos à configuração dos pinos de acordo com os *inputs* e *outputs* da nossa ALU. Uma vez que este modelo possui apenas dez *switches* e dez *LEDs*, e como nós temos 12 *inputs* para serem configurados e definidos na placa, tivemos de optar pelo uso dos dois botões *(buttons)*, presentes nesta mesma placa, para totalizar o número de entradas desejado.

Pin Assignments

```
set_global_assignment -name FAMILY "MAX 10 FPGA"
set_global_assignment -name DEVICE 10M50DAF484C6GES
set_global_assignment -name TOP_LEVEL_ENTITY "DE10_Lite"

# Switches
set_location_assignment PIN_C10 -to OP[0]
set_location_assignment PIN_C11 -to CBI
set_location_assignment PIN_D12 -to Y[0]
set_location_assignment PIN_C12 -to Y[1]
set_location_assignment PIN_A12 -to Y[2]
set_location_assignment PIN_B12 -to Y[3]
set_location_assignment PIN_A13 -to X[0]
set_location_assignment PIN_A14 -to X[1]
set_location_assignment PIN_B14 -to X[2]
set_location_assignment PIN_F15 -to X[3]

# LEDS
set_location_assignment PIN_A8 -to BE
set_location_assignment PIN_A9 -to GE
set_location_assignment PIN_A10 -to P
set_location_assignment PIN_B10 -to Z
set_location_assignment PIN_D13 -to OV
set_location_assignment PIN_C13 -to CBo
set_location_assignment PIN_E14 -to R[0]
set_location_assignment PIN_D14 -to R[1]
set_location_assignment PIN_A11 -to R[2]
set_location_assignment PIN_B11 -to R[3]

# BUTTONS
set_location_assignment PIN_B8 -to OP[2]
set_location_assignment PIN_A7 -to OP[1]
```

Figura 18: Código de atribuição de pinos

3.2 Testes para Operações Aritméticas

3.2.1 Teste 21 - Configuração e Resultado

Para a configuração do teste 21 da nossa tabela, optámos pela seguinte combinação de *inputs*:

$$X = 0101 \mid Y = 1001 \mid CBi = 1 \mid OP = 000$$

Resultado do teste:

$$R = 0110 \mid CBo = 0 \mid OV = 0 \mid Z = 0 \mid P = 0$$

! As *flags GE* (*Greater or Equal*) e *BE* (*Below or Equal*) não têm significado para a operação em questão.

Figura 19: Teste de um incremento

3.2.2 Teste 22 - Configuração e Resultado

Para a configuração do teste 22 da nossa tabela, optámos pela seguinte combinação de *inputs*:

$$X = 1110 \mid Y = 0110 \mid CBi = 0 \mid OP = 011$$

Resultado do teste:

$$R = 1000 \mid CBo = 0 \mid OV = 0 \mid Z = 0 \mid P = 1 \mid GE = 0 \mid BE = 1$$

Figura 20: Teste de uma subtração

3.3 Testes para Operações Lógicas

3.3.1 Teste 23 - Configuração e Resultado

Para a configuração do teste 23 da nossa tabela, optámos pela seguinte combinação de *inputs*:

$$X = 1100 \mid Y = 0011 \mid CBi = 1 \mid OP = 101$$

Resultado do teste:

$$R = 1110 \mid CBo = 0 \mid Z = 0 \mid P = 1$$

! As *flags* OV (*Overflow*), GE (*Greater or Equal*) e BE (*Below or Equal*) não têm significado para a operação em questão.

Figura 21: Teste de um deslocamento aritmético à direita

3.3.2 Teste 24 - Configuração e Resultado

Para a configuração do teste 24 da nossa tabela, optámos pela seguinte combinação de *inputs*:

$$X = 1011 \mid Y = 0001 \mid CBi = 1 \mid OP = 110$$

Resultado do teste:

$$R = 0110 \mid CBo = 1 \mid Z = 0 \mid P = 0$$

! As *flags* OV (*Overflow*), GE (*Greater or Equal*) e BE (*Below or Equal*) não têm significado para a operação em questão.

Figura 22: Teste de um deslocamento lógico à esquerda

3.4 Comparação dos Resultados

3.4.1 Resultados Teóricos *versus* Experimentais

Resultados Teóricos *versus* Resultados Experimentais

	TLAB_2 - Resultados									Resultado Teórico									Resultado Experimental								
	#	OP	CBi	X(2)	X(ℕ)	X(ℤ)	Y(2)	Y(ℕ)	Y(ℤ)	R(2)	R(ℕ)	R(ℤ)	CBo	OV	Z	P	GE	BE	R(2)	R(ℕ)	R(ℤ)	CBo	OV	Z	P	GE	BE
	1	000	0	0000	0	0	1111	15	-1	0000	0	0	0	0	1	0	–	–	0000	0	0	0	0	1	0	1	1
	2	000	1	0000	0	0	1111	15	-1	0001	1	1	0	0	0	1	–	–	0001	1	1	0	0	0	1	1	0
	3	000	1	0111	7	7	0101	5	5	1000	8	-8	0	1	0	1	–	–	1000	8	-8	0	1	0	1	0	1
	4	000	1	1111	15	-1	1010	10	-6	0000	0	0	1	0	1	0	–	–	0000	0	0	1	0	1	0	1	1
	5	001	1	0000	0	0	1111	15	-1	1111	15	-1	1	0	0	0	–	–	1111	15	-1	1	0	0	0	0	1
	6	001	1	0001	1	1	0101	5	5	0000	0	0	0	0	1	0	–	–	0000	0	0	0	0	1	0	1	1
	7	001	1	1000	8	-8	1010	10	-6	0111	7	7	0	1	0	1	–	–	0111	7	7	0	1	0	1	1	0
	8	010	0	0001	1	1	1110	14	-2	1111	15	-1	0	0	0	0	–	–	1111	15	-1	0	0	0	0	0	1
	9	010	1	0001	1	1	1110	14	-2	0000	0	0	1	0	1	0	–	–	0000	0	0	1	0	1	0	1	1
	10	010	1	1000	8	-8	1111	15	-1	1000	8	-8	1	0	0	1	–	–	1000	8	-8	1	0	0	1	0	1
	11	010	1	0010	2	2	0101	5	5	1000	8	-8	0	1	0	1	–	–	1000	8	-8	0	1	0	1	0	1
	12	011	0	0011	3	3	0101	5	5	1110	14	-2	1	0	0	1	0	1	1110	14	-2	1	0	0	1	0	1
	13	011	0	1110	14	-2	1100	12	-4	0010	2	2	0	0	0	1	1	0	0010	2	2	0	0	0	1	1	0
	14	011	1	0110	6	6	0110	6	6	1111	15	-1	1	0	0	0	0	1	1111	15	-1	1	0	0	0	0	1
	15	011	1	1000	8	-8	0111	7	7	0000	0	0	0	1	1	0	1	1	0000	0	0	0	1	1	0	1	1
	16	100	1	0001	1	1	1100	12	-4	0000	0	0	1	–	1	0	–	–	0000	0	0	1	0	1	0	0	0
	17	100	0	1101	13	-3	1001	9	-7	0110	6	6	1	–	0	0	–	–	0110	6	6	1	0	0	0	0	0
	18	101	0	1010	10	-6	1111	15	-1	1101	13	-3	0	–	0	1	–	–	1101	13	-3	0	0	0	1	0	0
	19	110	0	1001	9	-7	0110	6	6	0010	2	2	1	–	0	1	–	–	0010	2	2	1	0	0	1	0	0
	20	111	1	1111	15	-1	0100	4	4	1011	11	-5	–	–	0	1	–	–	1011	11	-5	1	0	0	1	0	0
Aritm	**21**	000	1	0101	5	5	1001	9	-7	0110	6	6	0	0	0	0	–	–	0110	6	6	0	0	0	0	1	0
Aritm	**22**	011	0	1110	14	-2	0110	6	6	1000	8	-8	0	0	0	1	0	1	1000	8	-8	0	0	0	1	0	1
Logic	**23**	101	1	1100	12	-4	0011	3	3	1110	14	-2	0	–	0	1	–	–	1110	14	-2	0	0	0	1	0	0
Logic	**24**	110	1	1011	11	-5	0001	1	1	0110	6	6	1	–	0	0	–	–	0110	6	6	1	0	0	0	0	0

Figura 23: Tabela completa de resultados

3.4.2 Análise da Consistência e Erros

De modo geral, os resultados demonstraram uma consistência satisfatória nas fases de testes práticos e teóricos ao longo do desenvolvimento do projeto. As operações da ALU foram testadas em diferentes cenários, e os valores das *flags* corresponderam, na maioria dos casos, ao comportamento esperado.

Durante o processo de validação, pequenos erros surgiram, como ajustes de configuração nas *flags* e comportamentos inesperados em operações específicas. No entanto, esses erros foram identificados rapidamente e resolvidos com pequenas correções no código VHDL e ajustes na configuração da placa.

4 Conclusão

4.1 Considerações sobre o trabalho desenvolvido

Ao longo do trabalho, analisámos o funcionamento interno de uma ALU (*Arithmetic Logic Unit*), a sua estrutura e o modo como esta realiza operações aritméticas e lógicas — funções estas que estão na base de qualquer aparelho/dispositivo digital que lide com cálculos e decisões lógicas.

A ALU, afinal, reflete a interação entre a teoria da matemática e a lógica digital, que em conjunto, permitem a construção de sistemas capazes de realizar tarefas que variam desde operações simples até às mais complexas.

Em suma, ao estudar e entender, logica e objetivamente, uma ALU, percebemos como funciona cada módulo desta, e adquirindo como bases, alguns módulos de trabalhos anteriores, pudemos aplicar e organizar a forma e maneira como se comportavam as entradas e saídas de cada parte da nossa ALU, a modo de atingir o objetivo proposto pelo enunciado.

4.2 Discussão de Resultados

No que diz respeito aos resultados obtidos neste segundo trabalho, podemos afirmar que houve uma conclusão de resultados efetiva e que está de acordo com o esperado. Contudo, durante a realização deste projeto, notamos que, devido à complexidade da criação de cada módulo desta ALU, é provável que surjam algumas irregularidades no que diz respeito ao código VHDL. Mesmo assim, conseguimos ultrapassar cada falha que foi surgindo, alcançando assim, um módulo da ALU conforme previsto inicialmente.

Essa unidade é bastante importante, uma vez que possibilita a resolução de problemas e operações envolvendo lógica e aritmética.

De modo geral, foi essencial estabelecermos um conhecimento inicial sobre uma ALU para garantir uma melhor orientação e consistência ao longo de todo o trabalho. Além disso, a verificação de cada caso particular de uma operação, deve ser muito bem organizada, especialmente com o auxílio de uma *testbench* previamente configurada e bem estruturada, conforme o código.

Referências

- **draw.io**: Usado para a realização do esquema elétrico da ALU;
- **Overleaf Documentation**: Usada em recurso para desenvolvimento do documento em *LaTeX*;
- **Rice University *LaTeX* Mathematical Symbols**: Documento da Universidade Rice, no Texas, dedicado a demonstrar símbolos e técnicas de formatação em *LaTeX* menos usados e que nos foi muito útil ao desenvolvimento do documento;
- **UC Irvine Department of Mathematics *LaTeX* Math Symbols**: Documento da Universidade da Califórnia, na cidade de Irvine, que contém uma compilação de símbolos, gráficos, tabelas e entre outros muito úteis para o desenvolvimento deste documento em *LaTeX*;
- **Manual for the tcolorbox package**: Manual de utilização da biblioteca de *LaTeX* "tcolorbox"utilizada para o design das caixas;
- **Apresentação - Circuitos Combinatórios**: Documento disponibilizado pelo professor Mario Véstias na plataforma *Moodle*;
- **Apresentação - Sistemas de Numeração**: Documento disponibilizado pelo professor Mario Véstias na plataforma *Moodle*;
- **Apresentação - Aritmética Binária**: Documento disponibilizado pelo professor Mario Véstias na plataforma *Moodle*;
- **Apresentação - Circuitos Aritméticos**: Documento disponibilizado pelo professor Mario Véstias na plataforma *Moodle*.

A VHDL

A.1 Módulo Aritmético

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity AU is
5      port (
6          A: in std_logic_vector (3 downto 0);
7          B: in std_logic_vector (3 downto 0);
8          CBi: in std_logic;
9          OPau: in std_logic;
10         R: out std_logic_vector (3 downto 0);
11         CBo: out std_logic;
12         OV: out std_logic
13     );
14 end AU;
15
16 architecture arch_au of AU is
17
18 component ADDSUB is
19     port (
20         A: in std_logic_vector(3 downto 0);
21         B: in std_logic_vector(3 downto 0);
22         Cbi: in std_logic;
23         OP: in std_logic;
24         R: out std_logic_vector(3 downto 0);
25         B3: out std_logic;
26         CBo: out std_logic
27     );
28 end component;
29
30 component FLAGS is
31     port (
32         A3: in std_logic;
33         B3: in std_logic;
34         R3: in std_logic;
35         iCBo: in std_logic;
36         OVo: out std_logic;
37         CBo: out std_logic
38     );
39 end component;
40
41 signal B3_temp, iCBo_temp,
42        R3_temp: std_logic;
43
44 begin
45     A0: ADDSUB port map(
46         A => A,
47         B => B,
48         OP => OPau,
49         Cbi => CBi,
50         R(3) => R3_temp,
51         R(2) => R(2),
52         R(1) => R(1),
53         R(0) => R(0),
54         B3 => B3_temp,
55         CBo => iCBo_temp
56     );
57
58     F0: FLAGS port map(
59         A3 => A(3),
60         B3 => B3_temp,
61         iCBo => iCBo_temp,
62         R3 => R3_temp,
63         Ovo => OV,
64         CBo => CBo
65     );
66
67     R(3) <= R3_temp;
68 end arch_au;
```

Figura 24: Código do Módulo Aritmético

A.1.1 Somador/Subtrator

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity ADDSUB is
5      port (
6          A: in std_logic_vector(3 downto 0);
7          B: in std_logic_vector(3 downto 0);
8          Cbi: in std_logic;
9          OP: in std_logic;
10         R: out std_logic_vector(3 downto 0);
11         B3: out std_logic;
12         CBo: out std_logic
13     );
14     end ADDSUB;
15
16 architecture arch_addsub of ADDSUB is
17 component ADDER is
18     port (
19         A: in std_logic_vector(3 downto 0);
20         B: in std_logic_vector(3 downto 0);
21         C0: in std_logic;
22         S: out std_logic_vector(3 downto 0);
23         C4: out std_logic
24     );
25 end component;
26
27 signal Cox, Cix: std_logic;
28 signal Bx: std_logic_vector(3 downto 0);
29
30 begin
31     A0: ADDER port map(
32         A => A,
33         B => Bx,
34         C0 => Cix,
35         S => R,
36         C4 => Cox
37     );
38
39     Cix <= Cbi xor OP;
40     Bx(0) <= B(0) xor OP;
41     Bx(1) <= B(1) xor OP;
42     Bx(2) <= B(2) xor OP;
43     Bx(3) <= B(3) xor OP;
44     Cbo <= Cox xor OP;
45     B3 <= Bx(3);
46 end arch_addsub;
```

Figura 25: Código do Somador/Subtrator

A.1.2 Somador de 4 Bits

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity ADDER is
5      port (
6          A: in std_logic_vector(3 downto 0);
7          B: in std_logic_vector(3 downto 0);
8          C0: in std_logic;
9          S: out std_logic_vector(3 downto 0);
10         C4: out std_logic
11     );
12     end ADDER;
13
14 architecture arch_adder of ADDER is
15 component fulladder
16     port (
17         A: in std_logic;
18         B: in std_logic;
19         Cin: in std_logic;
20         R: out std_logic;
21         Cout: out std_logic
22     );
23 end component;
24
25 signal C1, C2, C3: std_logic;
26
27 begin
28     FA0: fulladder port map(
29         A => A(0),
30         B => B(0),
31         Cin => C0,
32         R => S(0),
33         Cout => C1
34     );
35
36     FA1: fulladder port map(
37         A => A(1),
38         B => B(1),
39         Cin => C1,
40         R => S(1),
41         Cout => C2
42     );
43
44     FA2: fulladder port map(
45         A => A(2),
46         B => B(2),
47         Cin => C2,
48         R => S(2),
49         Cout => C3
50     );
51
52     FA3: fulladder port map(
53         A => A(3),
54         B => B(3),
55         Cin => C3,
56         R => S(3),
57         Cout => C4
58     );
59
60 end arch_adder;
```

Figura 26: Código Somador de 4 Bits

A.1.3 Somador de 1 Bit

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity FULLADDER is
5      port (
6          A: in std_logic;
7          B: in std_logic;
8          Cin: in std_logic;
9          R: out std_logic;
10         Cout: out std_logic
11     );
12 end FULLADDER;
13
14 architecture logicfulladder of FULLADDER is
15 begin
16     R <= A xor B xor Cin;
17     Cout <= (A and B) or ((A xor B) and Cin);
18 end logicfulladder;
```

Figura 27: Código Somador de 1 Bit

A.1.4 Flags (AU)

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity FLAGS is
5      port (
6          A3: in std_logic;
7          B3: in std_logic;
8          R3: in std_logic;
9          iCBo: in std_logic;
10         OVo: out std_logic;
11         CBo: out std_logic
12     );
13 end FLAGS;
14
15 architecture arch_flags of FLAGS is
16 begin
17     OVo <= (not A3 and not B3 and R3) or
18            (A3 and B3 and not R3);
19     CBo <= iCBo;
20 end arch_flags;
```

Figura 28: Código Flags (AU)

A.2 Módulo Lógico

```
1   library ieee;
2   use ieee.std_logic_1164.all;
3
4   entity LU is
5       port(
6           A, B: in std_logic_vector(3 downto 0);
7           S: in std_logic_vector(1 downto 0);
8           R: out std_logic_vector(3 downto 0);
9           CY: out std_logic
10      );
11  end LU;
12
13  architecture arc_lu of LU is
14
15  component LSR
16      port(
17          A: in std_logic_vector(3 downto 0);
18          O: out std_logic_vector(3 downto 0);
19          CY: out std_logic
20      );
21  end component;
22
23  component ASR
24      port(
25          A: in std_logic_vector(3 downto 0);
26          O: out std_logic_vector(3 downto 0);
27          CY: out std_logic
28      );
29  end component;
30
31  component LSL
32      port(
33          A: in std_logic_vector(3 downto 0);
34          O: out std_logic_vector(3 downto 0);
35          CY: out std_logic
36      );
37  end component;
38
39  component NANDL
40      port(
41          A, B: in std_logic_vector(3 downto 0);
42          O: out std_logic_vector(3 downto 0)
43      );
44  end component;
45
46  component MUX_LU
47      port(
48          A, B: in std_logic_vector(3 downto 0);
49          S: in std_logic;
50          Y: out std_logic_vector(3 downto 0)
51      );
52  end component;
53
54  signal o_LSR, o_ASR, o_LSL,
55          o_NAND, o_MUXa,
56          o_MUXb : std_logic_vector(3 downto 0);
57  signal o_CYa, o_CYb, o_CYc: std_logic;
58  begin
59
60      LSR0: LSR port map(
61          A => A,
62          O => o_LSR,
63          CY => o_CYa
64      );
65
66      ASR0: ASR port map(
67          A => A,
68          O => o_ASR,
69          CY => o_CYb
70      );
71
72      LSL0: LSL port map(
73          A => A,
74          O => o_LSL,
75          CY => o_CYc
76      );
77
78      NANDL0: NANDL port map(
79          A => A,
80          B => B,
81          O => o_NAND
82      );
83
84      UMUX0: MUX_LU port map (
85          A => o_LSR,
86          B => o_ASR,
87          S => S(0),
88          Y => o_MUXa
89      );
90
91      UMUX1: MUX_LU port map (
92          A => o_LSL,
93          B => o_NAND,
94          S => S(0),
95          Y => o_MUXb
96      );
97
98      UMUX2: MUX_LU port map (
99          A => o_MUXa,
100         B => o_MUXb,
101         S => S(1),
102         Y => R
103     );
104
105     CY <= (not S(1) and (o_CYa or o_CYb)) or
106           (S(1) and o_CYc);
107
108 end arc_lu;
```

Figura 29: Código do Módulo Lógico

A.2.1 ASR

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity ASR is
5  port(
6      A: in std_logic_vector (3 downto 0);
7      O: out std_logic_vector (3 downto 0);
8      CY: out std_logic
9  );
10 end ASR;
11
12 architecture arc_asr of ASR is
13
14 begin
15     CY <= A(0);
16     O(0) <= A(1);
17     O(1) <= A(2);
18     O(2) <= A(3);
19     O(3) <= A(3);end arc_asr;
```

Figura 30: Código do ASR

A.2.2 LSR

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity LSR is
5      port(
6          A: in std_logic_vector (3 downto 0);
7          O: out std_logic_vector (3 downto 0);
8          CY: out std_logic
9      );
10 end LSR;
11
12 architecture arc_lsr of LSR is
13
14 begin
15     CY <= A(0);
16     O(0) <= A(1);
17     O(1) <= A(2);
18     O(2) <= A(3);
19     O(3) <= '0';
20 end arc_lsr;
```

Figura 31: Código LSR

A.2.3 LSL

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity LSL is
5      port(
6          A: in std_logic_vector (3 downto 0);
7          O: out std_logic_vector (3 downto 0);
8          CY: out std_logic
9      );
10 end LSL;
11
12 architecture arc_lsl of LSL is
13
14 begin
15     O(0) <= '0';
16     O(1) <= A(0);
17     O(2) <= A(1);
18     O(3) <= A(2);
19     CY <= A(3);
20 end arc_lsl;
```

Figura 32: Código LSL

A.2.4 NAND

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity NANDL is
5      port(
6          A, B: in std_logic_vector(3 downto 0);
7          O: out std_logic_vector(3 downto 0)
8      );
9  end NANDL;
10
11 architecture arc_nandl of NANDL is
12
13 begin
14     O(0) <= A(0) nand B(0);
15     O(1) <= A(1) nand B(1);
16     O(2) <= A(2) nand B(2);
17     O(3) <= A(3) nand B(3);
18 end arc_nandl;
```

Figura 33: Código NAND

A.2.5 MUX (LU)

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity MUX_LU is
5      port(
6          A, B: in std_logic_vector(3 downto 0);
7          S: in std_logic;
8          Y: out std_logic_vector(3 downto 0)
9      );
10 end MUX_LU;
11
12 architecture arc_mux_lu of MUX_LU is
13
14 begin
15     Y(0) <= (not S and A(0)) or (S and B(0));
16     Y(1) <= (not S and A(1)) or (S and B(1));
17     Y(2) <= (not S and A(2)) or (S and B(2));
18     Y(3) <= (not S and A(3)) or (S and B(3));
19
20 end arc_mux_lu;
```

Figura 34: Código MUX (LU)

A.3 Decoder

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity DECODER is
5      port (
6          OP0, OP1, OP2: in std_logic;
7          OPA, OPB, OPC,
8          OPD, OPE, OPF: out std_logic
9      );
10 end DECODER;
11
12 architecture arch_decoder of DECODER is
13
14 begin
15     OPA <= OP1;
16     OPB <= OP0;
17     OPC <= OP2;
18     OPD <= OP0;
19     OPE <= OP1;
20     OPF <= OP2;
21
22 end arch_decoder;
```

Figura 35: Código Decoder

A.4 Flags (ALU)

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity FLAGS_ALU is
5      port (
6          iOV: in std_logic;
7          iCB: in std_logic;
8          OP: in std_logic;
9          R: in std_logic_vector(3 downto 0);
10         CY: in std_logic;
11         BE: out std_logic;
12         GE: out std_logic;
13         P: out std_logic;
14         Z: out std_logic;
15         OV: out std_logic;
16         CB: out std_logic
17     );
18 end FLAGS_ALU;
19
20 architecture arch_flags of FLAGS_ALU is
21
22 signal sZ: std_logic;
23
24 begin
25     BE <= (R(3) or sZ) and not OP;
26     GE <= (not R(3) or sZ) and not OP;
27     P <= (R(0) xor R(1)) xor (R(2) xor R(3));
28     sZ <= not (R(3) or R(2) or R(1) or R(0));
29     OV <= iOV and not OP;
30     CB <= (not OP and iCB) or (OP and CY);
31     Z <= sZ;
32 end arch_flags;
```

Figura 36: Código Flags (ALU)

A.5 Yor0

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity YOR0 is
5      port (
6          Y: in std_logic_vector (3 downto 0);
7          OPa: in std_logic;
8          B: out std_logic_vector (3 downto 0)
9      );
10 end YOR0;
11
12 architecture arch_Yor0 of YOR0 is
13
14 begin
15     B(0) <= Y(0) and OPa;
16     B(1) <= Y(1) and OPa;
17     B(2) <= Y(2) and OPa;
18     B(3) <= Y(3) and OPa;
19
20 end arch_Yor0;
```

Figura 37: Código Yor0

A.6 MUX (ALU)

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity MUX_ALU is
5      port(
6          A, B: in std_logic_vector (3 downto 0);
7          S: in std_logic;
8          Y: out std_logic_vector (3 downto 0)
9      );
10 end MUX_ALU;
11
12 architecture arc_mux_alu of MUX_ALU is
13
14 begin
15     Y(0) <= (not S and A(0)) or (S and B(0));
16     Y(1) <= (not S and A(1)) or (S and B(1));
17     Y(2) <= (not S and A(2)) or (S and B(2));
18     Y(3) <= (not S and A(3)) or (S and B(3));
19
20 end arc_mux_alu;
```

Figura 38: Código MUX (ALU)

A.7 ALU

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity ALU is
5  port (
6      CBi: in std_logic;
7      X: in std_logic_vector(3 downto 0);
8      Y: in std_logic_vector(3 downto 0);
9      OP: in std_logic_vector(2 downto 0);
10     R: out std_logic_vector(3 downto 0);
11     CBo: out std_logic;
12     OV: out std_logic;
13     P: out std_logic;
14     Z: out std_logic;
15     GE: out std_logic;
16     BE: out std_logic);
17 end ALU;
18
19 architecture arc_alu of ALU is
20
21 component YORO
22     port (
23         Y: in std_logic_vector(3 downto 0);
24         OPa: in std_logic;
25         B: out std_logic_vector(3 downto 0)
26     );
27     end component;
28
29 component AU
30     port (
31         A: in std_logic_vector(3 downto 0);
32         B: in std_logic_vector(3 downto 0);
33         CBi: in std_logic;
34         OPau: in std_logic;
35         R: out std_logic_vector(3 downto 0);
36         CBo: out std_logic;
37         OV: out std_logic
38     );
39 end component;
40
41 component DECODER
42     port (
43         OP0, OP1, OP2: in std_logic;
44         OPA, OPB, OPC,
45         OPD, OPE, OPF: out std_logic
46     );
47 end component;
48
49 component LU
50     port (
51         A, B: in std_logic_vector(3 downto 0);
52         S: in std_logic_vector(1 downto 0);
53         R: out std_logic_vector(3 downto 0);
54         CY: out std_logic
55     );
56 end component;
57
58 component FLAGS_ALU
59     port (
60         iOV: in std_logic;
61         iCB: in std_logic;
62         OP: in std_logic;
63         R: in std_logic_vector(3 downto 0);
64         CY: in std_logic;
65         BE: out std_logic;
66         GE: out std_logic;
67         P: out std_logic;
68         Z: out std_logic;
69         OV: out std_logic;
70         CB: out std_logic
71     );
72 end component;
73
74 component MUX_ALU
75     port (
76         A, B: in std_logic_vector(3 downto 0);
77         S: in std_logic;
78         Y: out std_logic_vector(3 downto 0)
79     );
80 end component;
```

Figura 39: Código ALU (Parte 1)

```
81
82      signal sOPA, sOPB, sOPC,
83             sOPD, sOPE, sOPF,
84             sCBO, sOV, sP,
85             sZ, sGE, sBE,
86             sCY, aluCBo, aluOV: std_logic;
87
88      signal sYORO, sLU, aluR,
89             sR: std_logic_vector(3 downto 0);
90
91  begin
92      DECODER0: DECODER port map(
93          OP0 => OP(0),
94          OP1 => OP(1),
95          OP2 => OP(2),
96          OPA => sOPA,
97          OPB => sOPB,
98          OPC => sOPC,
99          OPD => sOPD,
100         OPE => sOPE,
101         OPF => sOPF
102     );
103
104     YORO0: YORO port map(
105         Y => Y,
106         OPa => sOPA,
107         B => sYORO
108     );
109
110     AU0: AU port map(
111         A => X,
112         B => sYORO,
113         CBi => CBi,
114         OPau => sOPB,
115         R => aluR,
116         CBo => aluCBo,
117         OV => aluOV
118     );
119
120     LU0: LU port map(
121         A => X,
122         B => Y,
123         S(0) => sOPD,
124         S(1) => sOPE,
125         R => sLU,
126         CY => sCY
127     );
128
129     MUX0: MUX_ALU port map(
130         A => aluR,
131         B => sLU,
132         S => sOPC,
133         Y => sR
134     );
135
136     FLAGS0: FLAGS_ALU port map(
137         iOV => aluOV,
138         iCB => aluCBo,
139         OP => sOPF,
140         R => sR,
141         CY => sCY,
142         BE => sBE,
143         GE => sGE,
144         P => sP,
145         Z => sZ,
146         OV => sOV,
147         CB => sCBO
148     );
149
150     R <= sR;
151     CBo <= sCBO;
152     OV <= sOV;
153     P <= sP;
154     Z <= sZ;
155     GE <= sGE;
156     BE <= sBE;
157
158 end arc_alu;
```

Figura 40: Código ALU (Parte 2)

A.8 Test bench

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.numeric_std.ALL;
4
5  entity ALU_tb is
6  end ALU_tb;
7
8  architecture behavioral of ALU_tb is
9  component ALU
10     port(
11         X,Y : in std_logic_vector(3 downto 0);
12         OP : in std_logic_vector(2 downto 0);
13         CBi : in std_logic;
14         R : out std_logic_vector(3 downto 0);
15         CBo : out std_logic;
16         OV : out std_logic;
17         Z : out std_logic;
18         P : out std_logic;
19         GE : out std_logic;
20         BE : out std_logic
21     );
22     end component;
23
24     signal X_tb, Y_tb : std_logic_vector(3 downto 0);
25     signal OP_tb : std_logic_vector(2 downto 0);
26     signal CBi_tb : std_logic;
27
28     signal R_uut_tb : std_logic_vector(3 downto 0);
29     signal CBo_uut_tb : std_logic;
30     signal OV_uut_tb : std_logic;
31     signal Z_uut_tb : std_logic;
32     signal P_uut_tb : std_logic;
33     signal GE_uut_tb : std_logic;
34     signal BE_uut_tb : std_logic;
35
36 begin
37     UUT : ALU port map(
38                         X => X_tb,
39                         Y => Y_tb,
40                         OP => OP_tb,
41                         CBi => CBi_tb,
42                         R => R_uut_tb,
43                         CBo => CBo_uut_tb,
44                         OV => OV_uut_tb,
45                         Z => Z_uut_tb,
46                         P => P_uut_tb,
47                         GE => GE_uut_tb,
48                         BE => BE_uut_tb
49                     );
50
51     stimulus_generator: process
52     begin
53         --1
54         X_tb <= "0000";
55         Y_tb <= "1111";
56         CBi_tb <= '0';
57         OP_tb <= "000";
58
59         wait for 10 ns;
60
61         --2
62         X_tb <= "0000";
63         Y_tb <= "1111";
64         CBi_tb <= '1';
65         OP_tb <= "000";
66
67         wait for 10 ns;
68
69         --3
70         X_tb <= "0111";
71         Y_tb <= "0101";
72         CBi_tb <= '1';
73         OP_tb <= "000";
74
75         wait for 10 ns;
76
77         --4
78         X_tb <= "1111";
79         Y_tb <= "1010";
80         CBi_tb <= '1';
81         OP_tb <= "000";
82
```

Figura 41: Código Test Bench (Parte 1)

```
        wait for 10 ns;

        --5
        X_tb <= "0000";
        Y_tb <= "1111";
        CBi_tb <= '1';
        OP_tb <= "001";

        wait for 10 ns;

        --6
        X_tb <= "0001";
        Y_tb <= "0101";
        CBi_tb <= '1';
        OP_tb <= "001";

        wait for 10 ns;

        --7
        X_tb <= "1000";
        Y_tb <= "1010";
        CBi_tb <= '1';
        OP_tb <= "001";

        wait for 10 ns;

        --8
        X_tb <= "0001";
        Y_tb <= "1110";
        CBi_tb <= '0';
        OP_tb <= "010";

        wait for 10 ns;

        --9
        X_tb <= "0001";
        Y_tb <= "1110";
        CBi_tb <= '1';
        OP_tb <= "010";

        wait for 10 ns;

        --10
        X_tb <= "1000";
        Y_tb <= "1111";
        CBi_tb <= '1';
        OP_tb <= "010";

        wait for 10 ns;

        --11
        X_tb <= "0010";
        Y_tb <= "0101";
        CBi_tb <= '1';
        OP_tb <= "010";

        wait for 10 ns;

        --12
        X_tb <= "0011";
        Y_tb <= "0101";
        CBi_tb <= '0';
        OP_tb <= "011";

        wait for 10 ns;

        --13
        X_tb <= "1110";
        Y_tb <= "1100";
        CBi_tb <= '0';
        OP_tb <= "011";

        wait for 10 ns;

        --14
        X_tb <= "0110";
        Y_tb <= "0110";
        CBi_tb <= '1';
        OP_tb <= "011";

        wait for 10 ns;

```

Figura 42: Código Test Bench (Parte 2)

```
        --15
        X_tb <= "1000";
        Y_tb <= "0111";
        CBi_tb <= '1';
        OP_tb <= "011";

        wait for 10 ns;

        --16
        X_tb <= "0001";
        Y_tb <= "1100";
        CBi_tb <= '1';
        OP_tb <= "100";

        wait for 10 ns;

        --17
        X_tb <= "1101";
        Y_tb <= "1001";
        CBi_tb <= '0';
        OP_tb <= "100";

        wait for 10 ns;

        --18
        X_tb <= "1010";
        Y_tb <= "1111";
        CBi_tb <= '0';
        OP_tb <= "101";

        wait for 10 ns;

        --19
        X_tb <= "1001";
        Y_tb <= "0110";
        CBi_tb <= '0';
        OP_tb <= "110";

        wait for 10 ns;

        --20
        X_tb <= "1111";
        Y_tb <= "0100";
        CBi_tb <= '1';
        OP_tb <= "111";

        wait for 10 ns;

        --21 nosso
        X_tb <= "0101";
        Y_tb <= "1001";
        CBi_tb <= '1';
        OP_tb <= "000";

        wait for 10 ns;

        --22 nosso
        X_tb <= "1110";
        Y_tb <= "0110";
        CBi_tb <= '0';
        OP_tb <= "011";

        wait for 10 ns;

        --23 nosso
        X_tb <= "1100";
        Y_tb <= "0011";
        CBi_tb <= '1';
        OP_tb <= "101";

        wait for 10 ns;

        --24 nosso
        X_tb <= "1011";
        Y_tb <= "0001";
        CBi_tb <= '1';
        OP_tb <= "110";

        wait for 10 ns;

        wait;

    end process;
End behavioral;
```

Figura 43: Código Test Bench (Parte 3)

B Esquema Elétrico

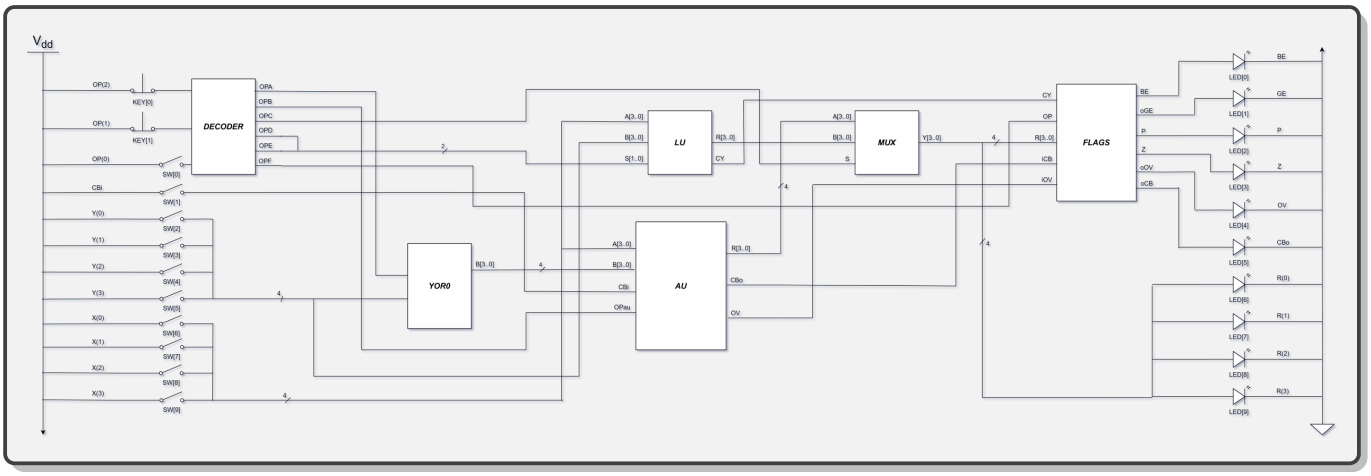

Figura 44: Esquema elétrico da ALU

Circuitos Aritméticos e Lógicos

Relatório do 2º Trabalho

Desenvolvimento de uma Unidade Aritmética e Lógica em VHDL

Este trabalho apresenta o desenvolvimento de uma ALU (*Arithmetic Logic Unit*) em VHDL, com foco na implementação de operações aritméticas e lógicas básicas em 4 bits. A ALU foi projetada e testada na placa DE10-Lite da *Intel*.

Através deste projeto, procura-se não apenas entender o funcionamento deste tipo de unidade, mas também aplicar teorias da computação e lógica digital, num ambiente prático de desenvolvimento...

Trabalho realizado por:

Gustavo Costa | Nº 52808

Rafael Pereira | Nº 52880

Turma: **LEIC15D**

Docente: **Prof.º Mário Véstias**

Licenciatura em Engenharia Informática e de Computadores

Lógica e Sistemas Digitais (LSD)

2024 / 2025 - Semestre de Inverno